

ISSEM Final Project

Matthew Bentz - mb9661@nyu.edu

1. Introduction

For more detailed information on LLM usage, source code fixes, vulnerability tests, or any other aspects of the project, please refer to the GitHub repository located at:

- <https://github.com/MatthewBentz/cs-gy-6803-final>

Below is a list of vulnerabilities that my group (group 13) found, validated, and tested against for the written assignment and lab submissions.

- Multiple commands sent after LOGOUT, parsed and executed as authenticated
 - Return was added after the LOGOUT command
- Client denial of service via timeout from SampleNetworkClient.py
 - Try/Catch with set timeouts were added to client calls
- Password was hardcoded
 - Added as an environment variable
- Session token list was unbounded
 - Added a bound of 32 maximum session tokens

Below is a list of vulnerabilities that do not apply regarding the scope of this project.

- SQL Injection
 - Select query was parameterized
- Hardcoded nonce value for password encryption
 - Random nonce was used for encryption

SQL injection does not apply to this project, as app.py was not included in the reference source and is deemed out of scope. Hardcoded nonce values are also out of scope, since they were included in the app.py and database.

Main Objectives:

- Validate LLM or tool-assisted vulnerabilities against running code
- Conduct a comprehensive security review against the product

Project Scope:

- <https://github.com/rajan0112/Infant-Incubator-Simulator>

2. Part 1 - LLM-Assisted Vulnerability Discovery

Vulnerability Table

Vuln	Affected Component	Description	CWE	Severity	How Discovered	How Validated	Affected Requirement	Recommended Fix
1	Authentication - run() / processCommands()	An attacker can inject a compound request to execute authenticated requests without a valid password or token	288 - CWE - CWE-288: Authentication Bypass Using an Alternate Path or Channel (4.20)	Critical	Manual	PoC + Screenshot - see below	1 - The system shall require a user to log in with a password before changing settings	Update parsing of commands in the server
2	Network Server (UDP sockets)	An attacker can read all communication to the server in cleartext	319 - CWE - CWE-319: Cleartext Transmission of Sensitive Information (4.20)	Critical	LLM + Manual	Trace + Screenshot - see below	4 - The system shall protect communication so it cannot be read or changed by outsiders	Implement and require SSL for server communication
3	SampleNetworkServer.py Line 107: msg = msg.decode("utf-8").strip()	An attacker can crash the server by sending malformed non-UTF data packets	248 - CWE - CWE-248: Uncaught Exception (4.20)	Critical	LLM	PoC + Screenshot - see below	5 - The system shall continue operating without stopping for more than 2 seconds	Guard incoming network commands with exception handling, implement zero-trust

Root Cause and Realistic Impact

Vulnerability #1:

The root cause for this vulnerability is a flaw in logic when parsing commands from the network (which could be the client or more likely a malicious actor on the same network). Any request containing two values formatted and sent as '[AUTH|LOGOUT] {Value2}' will be sent as privileged and executed as such, completely bypassing the requirement for an authenticated user. All commands on the server can be executed by chaining semicolons (;).

The realistic impact in the context of a network connected medical device is critical, essentially leaving the server wide open to tampering. An attacker could update the temperature unit or flood the server with requests to update the temperature, which could result in a worker taking action on incorrect values or being completely locked out of monitoring the infant.

Vulnerability #2:

The root cause for this vulnerability is a complete lack of encryption within the servers communication channels. Every authentication request, command, or otherwise information about the inner workings between the client and server are trivially known to an attacker on the same network, proved by the below screenshot. There are no protections in place for maintaining valid authentication, command execution, or keeping information confidential.

The realistic impact on a network connected medical device is that an attacker can quickly and easily identify valid credentials, execute protected commands, and otherwise gain complete information on the server's dependencies and functionality.

Vulnerability #3:

The root cause for this vulnerability is incorrect exception handling in the main thread loop for the SmartNetworkThermometer. When a packet is sent to the server that cannot be decoded into UTF-8, the decoder will throw a UnicodeDecodeError that is uncaught, causing the program to halt.

The realistic impact in the context of a network connected medical device is that the thermometer providing temperature readings stops completely. Both thermometers, the incubator and the infant thermometers, are affected by this vulnerability. This will result in the incubator/heater to stop heating or continue heating indefinitely using stale temperature data. The safety of the infant is compromised and will result in serious injury or death.

Vulnerability Assessment and Validation

Vulnerability #1: Unauthenticated command execution

The server is started and the below screenshot validates the vulnerability via the server's responses to the unauthenticated requests.

```
(.venv) mattybeanz@mattybeanz-pc:~$ echo "AUTH this_password_is_incorrect;GET_TEMP;SET_DEGF" | nc -u localhost 23456
312.213258
^C
(.venv) mattybeanz@mattybeanz-pc:~$ echo "AUTH this_password_is_incorrect;GET_TEMP" | nc -u localhost 23456
102.767121
```

Vulnerability #2: Cleartext network transmission

The left pane shows a valid authentication request and token response. The right shows a network capture of these two transmissions.

```
mattybeanz@mattybeanz-pc:~$ echo "AUTH real-password" | nc -u localhost 23456
33zpa4SE4tVf0aEY

mattybeanz@mattybeanz-pc:~$ sudo tcpdump -i lo udp port 23456 -n -X --print
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on lo, link-type EN10MB (Ethernet), snapshot length 262144 bytes
13:31:09.591597 IP 127.0.0.1.34470 > 127.0.0.1.23456: UDP, length 19
0x0000: 4500 002f 09ea 4000 4011 32d2 7f00 0001  E.../..@.2.....
0x0010: 7f00 0001 86a6 5ba0 001b fe2e 4155 5448  .....[.....AUTH
0x0020: 2072 6561 6c2d 7061 7373 776f 7264 0a    .real-password.
13:31:09.632951 IP 127.0.0.1.23456 > 127.0.0.1.34470: UDP, length 16
0x0000: 4500 002c 461f 4000 4011 f69f 7f00 0001  E...F.@. ....
0x0010: 7f00 0001 5ba0 86a6 0018 fe2b 3333 7a70  ....[.....+33zp
0x0020: 6134 5345 3474 5666 3061 4559          a4SE4tVf0aEY
```

Vulnerability #3: Network server thermometer crash

This screenshot validates that the test successfully crashes the thermometer thread.

```
(.venv) mattybeanz@mattybeanz-pc:/mnt/c/Users/matth/Proton Drive/matthewbentz/My files/NYU/CS-GY 6803 ISSEM/Final$ AUTH_PASSWORD=real-password python3 src/SampleNetworkServer.py
SmartNetworkThermometer running on port 23456
SmartNetworkThermometer running on port 23457
Exception in thread Thread-2:
Traceback (most recent call last):
  File "/usr/lib/python3.12/threading.py", line 1073, in _bootstrap_inner
    self.run()
  File "/mnt/c/Users/matth/Proton Drive/matthewbentz/My files/NYU/CS-GY 6803 ISSEM/Final/src/SampleNetworkServer.py", line 107, in run
    msg = msg.decode("utf-8").strip()
          ~~~~~^~~~~~
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xff in position 0: invalid start byte
```

This screenshot shows the test output, and a subsequent run to double-validate the server has become unresponsive.

```
(.venv) mattybeanz@mattybeanz-pc:/mnt/c/Users/matth/Proton Drive/matthewbentz/My files/NYU/CS-GY 6803 ISSEM/Final$ python3 vulnerability-assessment/malformed-packet-dos.py
[*] Checking whether server is listening on localhost:23456...
[+] Server responded (12 bytes).
[*] Sending malformed packet...
[+] Malformed packet sent: ffffd
[*] Waiting 3 seconds...
[*] Sending a valid request to check server health...
[FAIL] Server stopped responding after the malformed packet.
[FAIL] The suspected DoS vulnerability was reproduced.
(.venv) mattybeanz@mattybeanz-pc:/mnt/c/Users/matth/Proton Drive/matthewbentz/My files/NYU/CS-GY 6803 ISSEM/Final$ python3 vulnerability-assessment/malformed-packet-dos.py
[*] Checking whether server is listening on localhost:23456...
[!] Server did not respond to the initial request.
[FAIL] Server is not responding before the test.
```

LLM False Positives

False Positives #1: Plaintext password comparison and timing attack

The LLM found that a byte-for-byte comparison results in a vulnerability to timing attacks. An attacker could measure response times to authentication requests to determine the correct password.

This vulnerability is ultimately non-exploitable in this codebase because the server does not respond to wrong password attempts.

A quick PoC was used to validate my belief, and as expected, each request for a random password (and an attempt to gather the response time as a metric for password cracking) failed with a timeout.

[illegible]

3. Part 2 - Final Security Analysis Report

0.) Background

The product under review is an infant incubator, designed to create a safe, ideal, and controlled environment for infants to live in while their vital organs develop. In its current state, the product contains 3 vital components - the incubator runtime to maintain temperature, a network connected thermometer to set and display temperature readings, and a client for workers to monitor the incubators state and function.

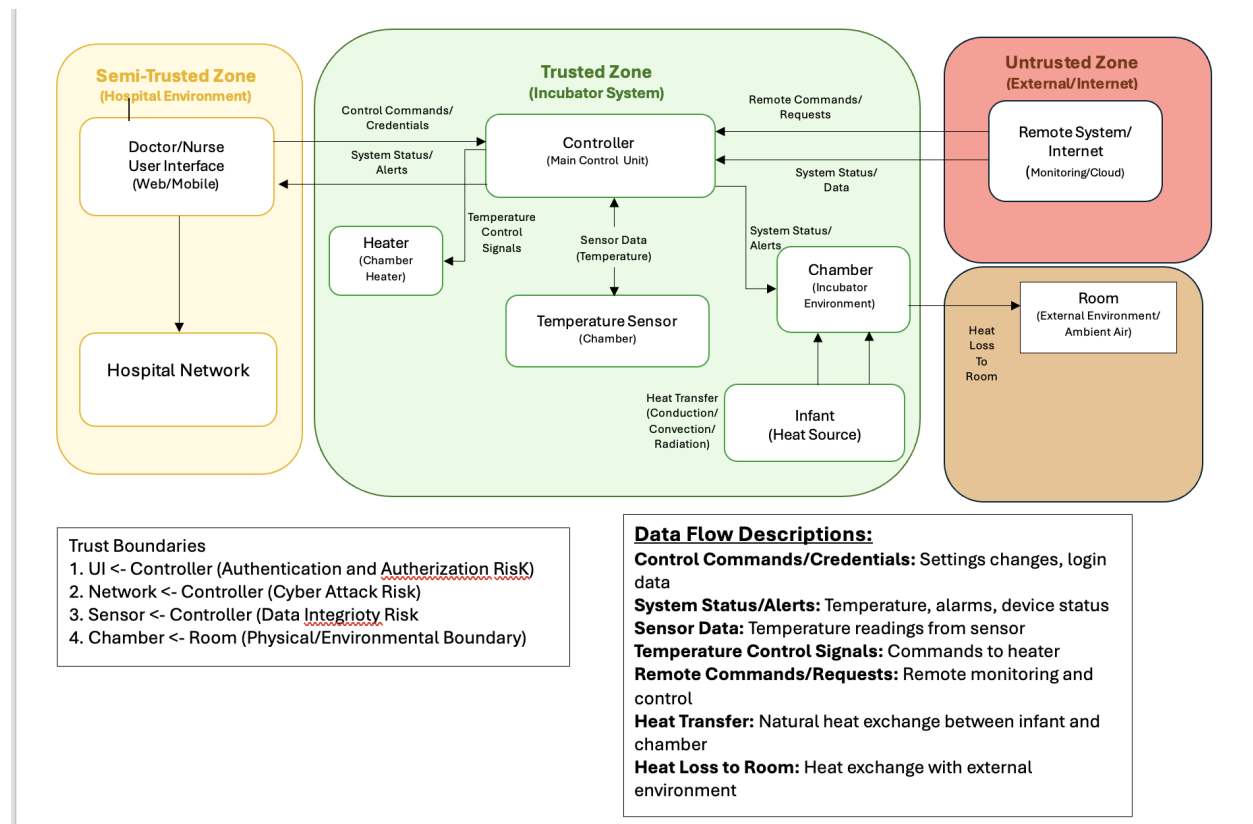
Security is important for this product because it directly affects the safety of the product. Lapses in security requirements for the incubator can create a dangerous environment for infants, resulting in potential injury or death. Safety of the infant is the top priority, which must be protected by a robust security implementation.

1.) Security Requirements for the Infant Incubator

ID	Requirement	Why it is Needed	How It Is Tested
1	The system shall require a user to log in with a username and password before changing settings	Prevents unauthorized people from changing incubator settings	Try accessing settings without login → access denied
2	The system shall lock a user account for 5 minutes after 5 failed login attempts within 10 minutes	Stops attackers from guessing passwords	Enter wrong password 5 times → account locks
3	The system shall check that temperature readings have not been altered before using them	Prevents unsafe decisions from incorrect data	Send incorrect data → system rejects it
4	The system shall protect communication so it cannot be read or changed by outsiders	Prevents data tampering and spying	Attempt unsecured connection → rejected
5	The system shall continue operating without stopping for more than 2 seconds	Ensures continuous safety for the infant	Simulate load → system remains active
6	The system shall keep temperature between 36°C and 37.5°C if sensors fail	Maintains safe environment during failure	Disconnect sensor → temperature remains stable

7	The system shall record all user actions with time and user ID	Helps track system usage and issues	Perform action → verify it is logged
8	The system shall only install verified software updates	Prevents harmful or fake updates	Install invalid update → rejected

2.) DFD for the Infant Incubator



3.) Asset List

Asset #	Asset Name	Asset Description	Concrete or Abstract Asset?	Critical Asset or not?
1	User Credentials	The username and password required to login and send commands	Abstract	Yes
2	Incubator Code	Software that authenticates, reads sensors, maintains temperature within the incubator	Abstract	Yes
3	Heater	Physical device that takes commands and	Concrete	Yes

		warms the incubator		
4	Temperature Data	Sensor values measured for the infant, chamber, and room	Abstract	Yes
5	Thermometer	Physical device that reads and transmits temperature data	Concrete	Yes
6	Audit Data	Record of access and temperature readings/commands (when applicable)	Abstract	No
7	Network	Connection between the incubator's server and client	Concrete	Yes

4.) Threat List

Threat ID	Asset	STRIDE Category	Attacker	Threat Description	Consequences of a successful attack
1	User Credentials	Elevation of Privilege	Hospital staff, visitor, malicious or non-authorized user client	An authenticated user performs high-risk functions such as setting the temperature	An attacker or uneducated user can issue unsafe commands, endangering the infant
2	Incubator Code	Tampering	Anyone/Anything with access to the server and client programs	Untrusted executables replace the original programs	Attackers could reprogram the server or client to target a different temperature, accept different auth
3	Heater	Tampering	Someone with physical access to the connected heater	The heater is covered, uncovered, unplugged, or not placed in its indented position	The heater may stop warming or overheat, causing harm to the infant
4	Temperature Data	Tampering	A malicious process or network connected device	Interception and modification of temperature data over the network	The incubator acts on false data by turning on or off, or the client shows false readings
5	Thermometer	Tampering	Someone with physical access to the connected thermometer	The thermometer is moved away from its intended position, reconfigured, or disconnected	The thermometer will send untrusted or no data, causing the controller to have unexpected behavior
6	Audit Data	Repudiation	Malicious process, user, or malfunctioning software	Authentication and command actions are not recorded or are stolen	After an incident, root cause analysis and remediation is impossible
7	Network	Denial of Service	Any network connected device with access to the server	Requests flood the server, causing it to crash or process legitimate requests	The incubator will cease to maintain the temperature by leaving the heater on or off for long periods of time
8	Incubator Code	Denial of Service	Bug in the server or client	The control process crashes or enters an infinite loop causing it to stop functioning	Alerts, logging, incoming commands are all halted and the incubator cannot maintain temperature

9	Network + User Credentials	Information Disclosure	A malicious process or network connected device	Authentication and command requests are intercepted on the network	Credentials, users, and operations are stolen and sold, retransmitted, or used in another attack
---	----------------------------	------------------------	---	--	--

5.) Risk Analysis

Previous full risk analysis can be found here:

[MatthewBentz/cs-gy-6803-lab4/lab4/writeup/Risk Assessment.pdf](https://www.matthewbentz.com/cs-gy-6803-lab4/lab4/writeup/Risk%20Assessment.pdf)

Risk #	Risk Name	Related Threat	Likelihood	Impact	Risk Level
1	Interception and modification of temperature data over the network	4	High	High	High
2	Authentication and command actions are not recorded or are stolen	6	Critical	Medium	Medium
3	The control process crashes or enters an infinite loop causing it to stop functioning	8	High	Critical	Critical
4	Authentication and command requests are intercepted on the network	9	High	Critical	Critical

Risk #1:

The likelihood an attacker intercepts and/or modifies temperature data over the network is high because the data is sent unencrypted (clear text). There are also no integrity checks to verify that the data is being sent from a valid process. It is trivial for an attacker on the network to conduct this attack.

The impact is rated at high because the temperature data of the infant/incubator shown in the client that workers use to monitor will either be missing or invalid.

The risk level is high because this affects the clients output and the workers actions directly. It is not marked as critical because it cannot be utilized to set the temperature reading internally to the physically connected incubator and endanger the safety of the infant.

Risk #2:

The likelihood of this risk is critical because no audit log data is kept.

The impact is medium because it is impossible to identify the historical state of the system, who has accessed the system, or otherwise conduct analysis on the system.

The risk is rated as medium because it does not affect the functionality, and it's possible to ascertain the state/access through other means such as network monitoring. Regardless, this risk can be easily remediated and should be included in the short-term road map.

Risk #3:

The likelihood of the control process crashing is high because we have identified and validated multiple points of failure for the system.

The impact is critical because an invalid state or complete crash can leave the infant in an unsafe environment.

The risk is critical because the multiple points of failure can come from malicious actors or even ignorant processes connected with the system. The resulting danger marks this as a must fix.

Risk #4:

The likelihood an attacker retrieves valid credentials and command information is high because no communication is encrypted. Again, it is trivial for an attacker to conduct this attack, however, the risk likelihood is focused mostly on interception of sensitive data only (and not modification).

The impact is rated as critical because an attacker can retrieve sensitive login information and map the functionality of the server. The credentials can be utilized to modify the state of the thermometer and client.

The risk is critical because unlike risk #1, a successful attack can control and modify the server's process without validation.

6.) Software Analysis

The following vulnerabilities are present in the current state of the infant incubator code. These additional vulnerabilities were identified and validated after the previous version patches.

Vuln	Affected Component	Description	Affected Requirement	Recommended Fix
1	Authentication - run() / processCommands()	An attacker can inject a compound request to execute authenticated requests without a valid password or token	1 - The system shall require a user to log in with a password before changing settings	Update parsing of commands in the server
2	Network Server (UDP sockets)	An attacker can read all communication to the server in cleartext	4 - The system shall protect communication so it cannot be read or changed by outsiders	Implement and require SSL for server communication

3	SampleNetworkServer Line 107: msg = msg.decode("utf-8").strip()	An attacker can crash the server by sending malformed non-UTF data packets	5 - The system shall continue operating without stopping for more than 2 seconds	Guard incoming network commands with exception handling, implement zero-trust
---	--	--	--	---

The following vulnerabilities were identified and are patched in the infant incubator's current state. These vulnerabilities have not been signed off from QA.

Vuln	Affected Component	Description	Affected Requirement	Patch Implemented
4	Authentication - LOGOUT	An attacker can chain server commands with a valid token in a LOGOUT request	1 - The system shall require a user to log in with a password before changing settings	Early return was added after a valid sessions LOGOUT action
5	SampleNetworkClient	A timeout or unreachable server causes the Client to freeze indefinitely	5 - The system shall continue operating without stopping for more than 2 seconds	Error handling was added in the Client for the server's request/response
6	User Credentials	Password used to authenticate was hard-coded in client/server programs	1 - The system shall require a user to log in with a password before changing settings	Password was added as a required environment variable
7	SampleNetworkServer	Session tokens were kept in-memory indefinitely, growing an unbounded list	5 - The system shall continue operating without stopping for more than 2 seconds	A maximum capacity of 32 was added for the list of in-memory session tokens

7.) Final Conclusion on Release

- (i). Launch as is, with recommended fixes to be pushed as patches in the field or in the near future
- (ii). Launch only after the recommended fixes are implemented and validated
- (iii). **Do a serious redesign and redevelopment and do this review again**

Unfortunately, this security analysis has resulted in the conclusion that a serious redesign and redevelopment of the infant incubator product is necessary. The identified and proven vulnerabilities associated with the current state of the product violate every requirement, and furthermore, are not a completely exhaustive list of issues. Infant safety is put in severe risk with the current design, and thus we cannot recommend this product fit for launch.

To understand our recommendation, we will outline the decision factors into three parts, confidentiality, integrity, and availability. Each section has a direct correlation to the previously consolidated lists for requirements, assets, threats, risks, and vulnerabilities.

Confidentiality

Confidentiality means keeping data secret. In the case of the infant incubator product, no data is kept secret. This is an important aspect for a network connected medical device that when used will contain regulated and protected sensitive information about an infant's health, a hospital's network, or a worker's credentials. Specifically, there is no encryption present for the network traffic from the thermometer server which exposes temperature readings, authentication requests, and processed commands. Anyone with access to the network can read, store, and reuse this information to control the incubator or violate privacy.

In addition, the password that is required to authenticate with the server (which was hardcoded) is now present in an environment variable. While this is an improvement, it is still injected at runtime, which means there is no way to validate that the user who is viewing/altering the incubator is proven to be authenticated or is checked to determine if they are authorized; i.e. a user accessing the system behind a valid, authorized, hospital worker. In reality, anyone with physical access to the device is completely capable of viewing/altering the temperature data or potentially the infant's data. Even if we were able to identify the user accessing the system, there is no mechanism in place to validate whether they are able to view or modify this data either, which strengthens our recommendation for redesign.

The lack of protection on the incubator's internal process, the network traffic, and the authentication mechanism result in a high risk for the infant's safety. Any physically present or network connected malicious actor is able to easily assess crucial functionality and identify multiple avenues for attacking the system. Without these protections in place, the incubator violates multiple requirements to prevent misuse and keep data confidential.

Integrity

Integrity means keeping systems safe from inaccurate and tampered information. The infant incubator has no checks to keep it safe. To start with, there are no tests that prove the functionality of the system. Without precise, exhaustive, and repeatable tests, there is no way to confidently assess how the incubator will function in a real world environment. The loss of sensors, high load, tampered data, or otherwise edge cases that can reveal how the system will react are not covered. This is an important aspect to maintaining a safe environment for the infant.

Another way the integrity of the incubator can be compromised is a vulnerability that bypasses the authentication mechanism. An attacker can send a crafted message to the server to execute commands that should require an authenticated user. Combined with the lack of encryption for these commands, this becomes an almost certain path for exploiting the system. Not only can the authentication be bypassed, it can be brute-forced, which means an attacker can try every password combination without failure or lockout. The product is wide open for malicious actors to gain access, break the system, and harm the infant.

Moreso, there are no checks currently in place that verify the process that is sending those commands. Anything, from anywhere, can pose as an approved user or process to read and execute commands. A hospital worker and a hacker are indistinguishable.

Lastly, audit data is not kept. If anyone accesses or modifies the system, there is no method to identify who viewed that data or took that action. Ensuring that we can verify that the system is functioning properly, only valid users are using the system, and we can see who took action against the system is important in maintaining its integrity. Without audit data, there is no way to find out what happened to the process or if it was tampered with.

Availability

Availability means keeping the system up and running. In its current state, the infant incubator is susceptible to multiple points of failure. Also, how it handles those failures is inadequate and presents a threat to the infant's safety. For example, if a thermometer stops reading the temperature, the incubator will continue operating with incorrect information. This is compounded by the fact that the temperature data is easily tampered with and the server that keeps this data is easily crashed. This could result in a heater continuously heating or failing to heat at all, endangering the life of the infant.

One of the many points of failure for the incubator is its lack of rate limiting. This means that the server will accept an unlimited amount of requests. This coincides with the risk that an attacker could brute-force password attempts, but it also can result in the server freezing or crashing due to running out of resources. This is a vital component of the incubator that needs to be assessed and remediated before release.

Another comes from the client, which shows the readings and current status of the incubator. There is brittle error handling, and failed requests can block the UI from updating - resulting in a hospital worker reacting to false assumptions or lacking knowledge about the true status. While timeout handling has been patched by the security team, there are still no methods in place to alert users or better handle these errors to prevent confusion.

Lastly, the server is critically vulnerable to crashing and failing completely with no recovery methods. This vulnerability was proven and can stem from a malicious actor as well as an error elsewhere in the system or even a benign and ignorant process on the hospital's network. This vulnerability, combined with the lack of tests, and the consequences of stale temperature data, result in a high risk scenario for malfunction and harm to the enclosed infant.

Requirements Unmet

Below are the list of product requirements and their associated vulnerabilities and reasons why they are unmet.

1. The system shall require a user to log in with a username and password before changing settings

This requirement is violated because it is trivial to spoof the current authentication mechanism. The user also does not supply a username/password when running the application, but rather it is inserted into the runtime through an environment variable. Because network traffic is not encrypted, it is also trivial for an attacker to retrieve valid credentials and log in successfully.

2. The system shall lock a user account for 5 minutes after 5 failed login attempts within 10 minutes

This requirement is violated because there is no rate limiting or lockout mechanism in place. An attacker can brute force a login through many attempts in a short period of time.

3. The system shall check that temperature readings have not been altered before using them

This requirement is violated because the system has no integrity checks. The data that is received from the thermometer has no message authentication, digital signature, or checksum. Any plaintext payload over the network received at the UDP ports will be accepted.

4. The system shall protect communication so it cannot be read or changed by outsiders

This requirement is violated because all network traffic is sent in the clear. Outsiders can read and change data with ease.

5. The system shall continue operating without stopping for more than 2 seconds

This requirement is violated because of the proven vulnerabilities that will halt the program from execution. The server can be crashed from non-unicode characters, and the client can be frozen from broken connections to the server. Each failure is not recorded or notified to the user.

6. The system shall keep temperature between 36°C and 37.5°C if sensors fail

This requirement is violated because the incubator will continue to use the last known temperature reading if the sensors fail. This is important because it creates a very dangerous environment for the infant, as the heaters will continue to heat or cease to heat.

7. The system shall record all user actions with time and user ID

This requirement is violated because the system contains no audit data. Nothing is logged or otherwise recorded. No historical data can be traced to a user.

8. The system shall only install verified software updates

This requirement is violated because the application is not cryptographically signed. The program can be replaced by attackers to achieve nefarious goals. Notably, there is also no update mechanism in the application.

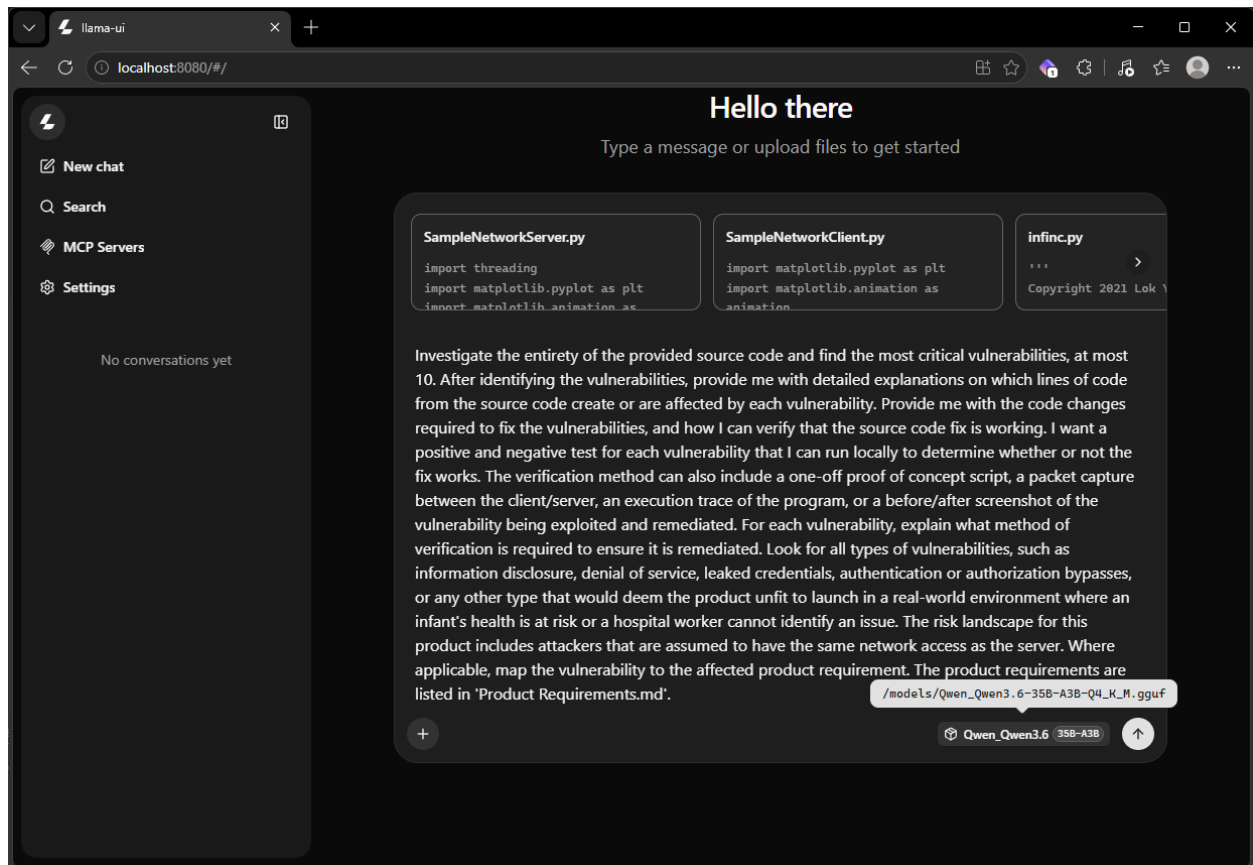
4. Appendix

Local llama.cpp with Qwen 3.6:

bartowski/Qwen_Qwen3.6-35B-A3B-GGUF

https://huggingface.co/bartowski/Qwen_Qwen3.6-35B-A3B-GGUF

- Specifically, the Q4_K_M quantized variant running at 128K context.



Sample output:

product unfit to launch in a real-world environment where an infant's health is at risk or a hospital worker cannot identify an issue. The risk landscape for this product includes attackers that are assumed to have the same network access as the server. Where applicable, map the vulnerability to the affected product requirement. The product requirements are listed in 'Product Requirements.md'.

Reasoning

Here is a comprehensive security analysis of the provided codebase, mapped to the Product Requirements, with remediation code, verification tests, and methodology.

1. Plaintext Password Comparison & Timing Attack

Maps to: PRD 1, 4

Location: SampleNetworkServer.py, if cs[1] == AUTH_PASSWORD.encode("utf-8") :

Explanation: The server compares the incoming password byte-for-byte using ==. This is vulnerable to timing attacks where an attacker can determine the correct password character-by-character by measuring response times. It also stores/compares passwords in plaintext.

Remediation:

PYTHON

```
import hmac
import hashlib
# In __init__: self.password_hash = hashlib.sha256(AUTH_PASSWORD.encode()).digest()
# In processCommands:
if hmac.compare_digest(hmac.new(self.password_hash, cs[1], hashlib.sha256).digest(), self.p
    # Auth success
```

Verification:

• Positive Test: Send AUTH <correct_hashed_input> → Returns valid token.

• Negative Test: Send AUTH <slightly_modified_input> → Returns Invalid password, response time matches correct password response (<1ms variance).

Verification Method: Run a timing attack script locally that sends thousands of AUTH requests with varying password prefixes and plots response times. Remediated code will show a flat line. Compare hmac.compare_digest behavior against ==.

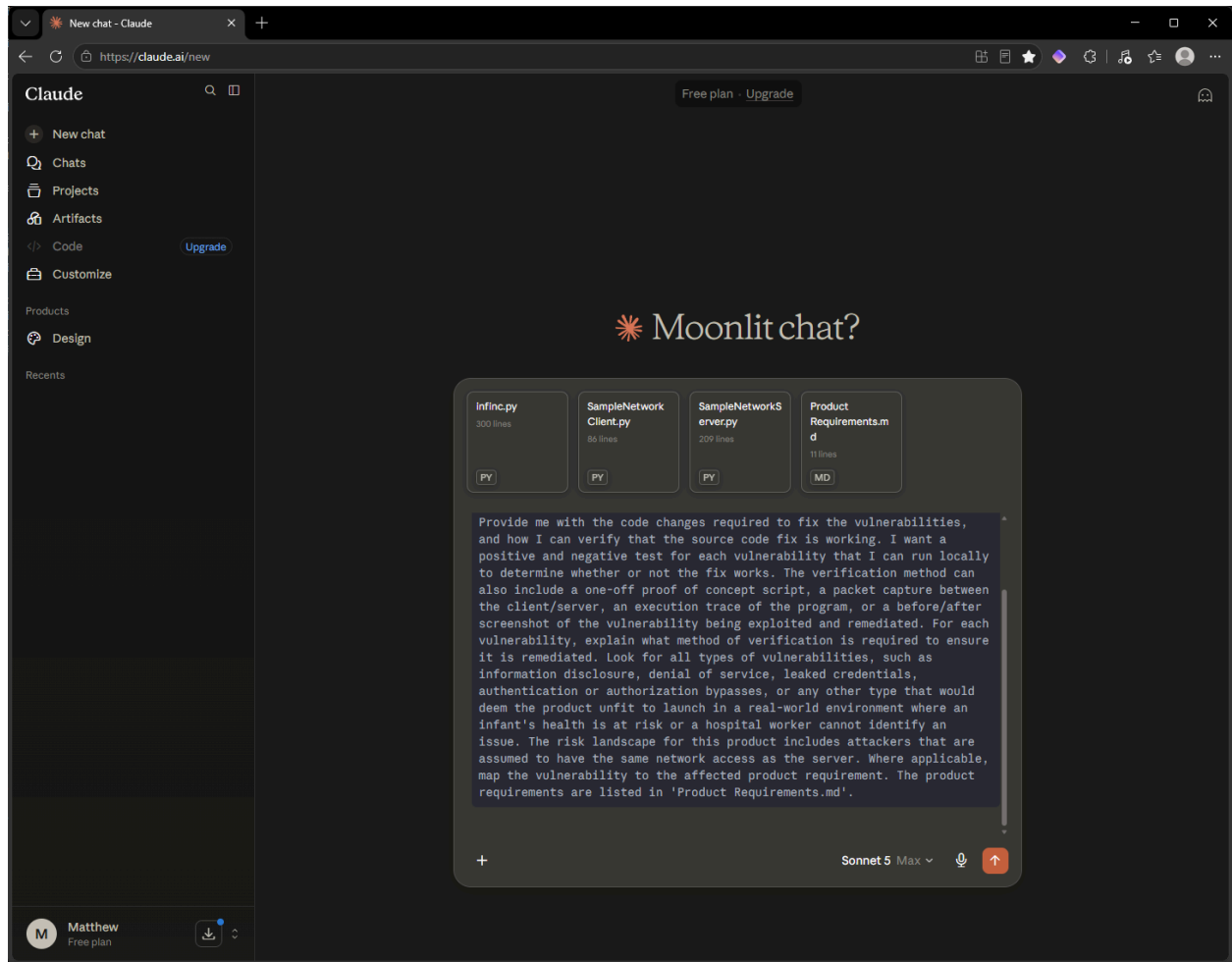
2. Absence of Account Lockout / Rate Limiting

Type a message...

Qwen_Qwen3.6 35B-A3B

Remediation:

Anthropic's Claude Sonnet 5 on Max reasoning:



One thing to note on Claude's output is that it stood up ephemeral client/server based on the provided code and verified the vulnerabilities. I will manually verify them as well, but thought that was a great validation step from the model.

vulnerability that I can run locally to determine whether or not

Show more

Analyzed security vulnerabilities in infant incubator code systematically

I'll start by thoroughly investigating this codebase. Given this is a network-connected infant incubator control system, I'm going to actually stand up the server and client in a sandbox and empirically verify each vulnerability I find — not just read the code — so the fixes and tests I give you are proven to work rather than theoretical.

Analyzed server process termination mechanics and shell behavior

That confirms it — background processes don't persist between tool calls, so I need to launch the server and run each test within a single shell invocation. Let me build proper test harnesses.

Pinpointed authentication bug causing indefinite request hangs

The AUTH bug is empirically confirmed — the server is alive, but a **correct** password behaves identically to a **wrong** one (silent timeout). Now let's test the thread-crash denial-of-service.

Identified critical thread-crash vulnerability enabling silent sensor failure

Confirmed — one malformed packet permanently kills a single sensor's reporting thread while the rest of the process (including the *other* sensor) keeps running normally, so nothing alarms. Now let's quantify the brute-force exposure before building fixes.

Architected comprehensive security hardening with encryption, authentication, and replay defenses

Now let's check what crypto tooling is available before designing the fix for the encryption/integrity issues.

Architected comprehensive cryptographic security overhaul addressing authentication and authorizat...

Good — **cryptography** is available. Now let me design a shared authenticated-encryption helper that will close both the cleartext-transmission and the response-spoofing vulnerabilities together, since they're really two sides of the same missing-transport-security gap.

Architected secure protocol redesign and fortified authentication mechanisms

Now the fixed server, integrating the crypto layer plus fixes for the exception-handling DoS, the broken auth comparison, missing lockout, weak PRNG, and missing audit logging all at once:

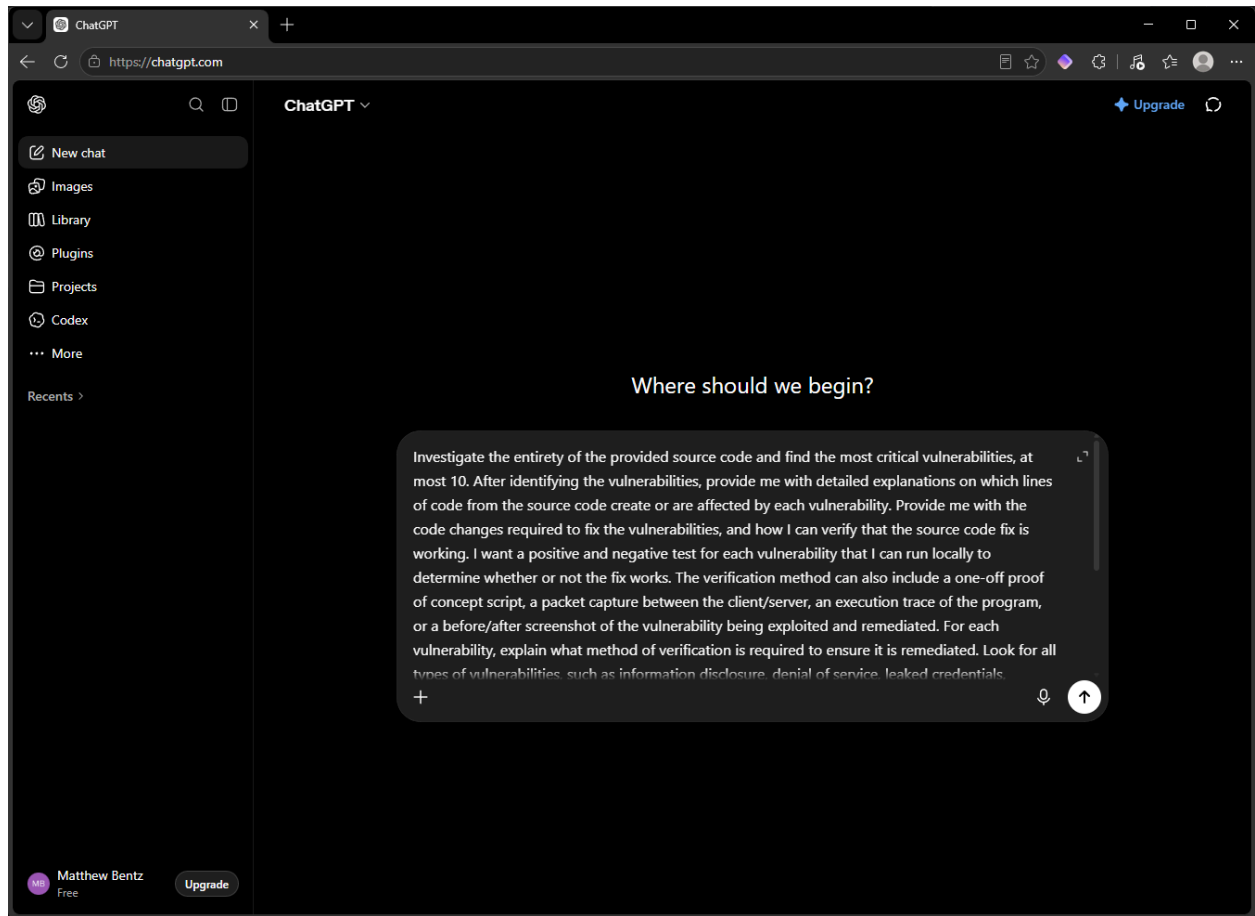
Hardened server: encrypted+authenticated transport, working auth comparison, account lockout, CSPR...



I stood up both the original and a hardened version of your server/client in a sandbox and empirically proved (not just read-and-guessed) 8 critical vulnerabilities:

1. **No integrity check on data in transit** — the client accepts *any* UDP packet as a genuine temperature reading. I proved this by standing up a rogue responder that never saw the password/token, and it fed the client a forged `999.9°` reading with zero resistance. (Req #3)
2. **Thread-crash DoS** — a single malformed (non-UTF-8) packet permanently kills the sensor thread via an uncaught `UnicodeDecodeError`. Confirmed live: server answers normally, I send one bad packet, and it goes silent forever — while the *other* thermometer's thread keeps working fine, so nothing looks wrong. This means stale data could silently drive the heater. (Req #5, #6)
3. **Cleartext transport** — password, session tokens, and temperature values are fully readable on the wire. Captured the literal bytes as proof. (Req #4)
4. **Authentication is completely broken** — `SampleNetworkServer.py` line 76 compares a `str` to `bytes`, which is *always* `False` in Python 3. I proved empirically that the correct password and a wrong password behave identically — nobody can ever log in, including legitimate hospital staff. (Req #1)
5. **No account lockout** — sent 200 distinct password guesses in 0.001s (~340,000/sec) with zero throttling. (Req #2)
6. **No audit logging** — zero log output exists anywhere in the original.
7. **Weak PRNG** (`random` instead of `secrets`) for session tokens — reproducible/predictable given seed state.
8. **Timing side-channel** in the password comparison (`==` instead of constant-time).

OpenAI's ChatGPT (does not show model used):



Sample output:

execution trace of the program, or a before/after screenshot of the vulnerability being exploited and remediated. For each vulnerability, [Show more](#) ▾

Worked for 32s >

I reviewed all four supplied files end-to-end. The biggest issue is that this is not just a matter of a few hardening fixes: **the current security model is insufficient for a real infant-care environment**. Several product requirements have no implementation at all, while the implemented network protocol has authentication, confidentiality, integrity, session-management, and availability weaknesses.

The most serious problems are below, ordered roughly by risk.

Edit

📄 ⬇ ↗

Security Review — Smart Network Thermometer / Incubator

Executive Summary

The supplied implementation should **not be considered launch-ready for a safety-critical hospital environment**.

The most significant findings are:

1. **Credentials and temperature traffic are transmitted in cleartext over UDP.**
2. **There is no username/account model and no brute-force lockout.**
3. **Authentication tokens are bearer credentials generated with a non-CSPRNG and remain valid for 24 hours.**
4. **A malformed network packet can terminate a thermometer's network thread.**
5. **Sensor failures can leave the system operating indefinitely on a stale temperature, including leaving the heater active.**
6. **Temperature data is not validated for plausibility or integrity before it drives safety decisions.**
7. **The GUI can become unresponsive for up to five seconds when network communication fails.**
8. **There is no security audit logging.**
9. **There is no verified software-update mechanism.**

The current server does bind only to `127.0.0.1`, which limits direct remote exploitation of the supplied sample. However, that also means the current implementation is not actually providing a network service to another machine. If the bind address is changed to a real network interface—as would be required for a distributed hospital deployment—the UDP security weaknesses become directly exploitable by any attacker with network access. The requirement explicitly assumes attackers can have the same network access as the server.

↓

The product requirements themselves make the gaps especially concerning:

+ Ask anything 🔊 🎧